Vídeo 5.2: O porteiro da API (middleware)

Duração-alvo: 6 min | **Conceito:** middleware e next() | **O aluno sai sabendo:** o que é um middleware, escrever um simples e saber o que acontece se esquecer o next().

Antes de gravar: partir do código do fim do módulo 4 (versão v4), com o servidor rodando e o requisicoes.http aberto.

[0:00] Abertura

Na tela: título do vídeo.

Fala: “No vídeo anterior vimos que a API precisa conferir os dados que chegam. Para fazer isso do jeito certo, precisamos conhecer uma ferramenta muito importante do Express: o middleware. É mais simples do que o nome sugere.”

[0:40] O porteiro

Na tela: ilustração da entrada de uma biblioteca, com um porteiro entre a porta e o balcão.

Fala: “Pense na entrada da biblioteca. Antes de chegar ao balcão, toda pessoa passa por um porteiro. O porteiro pode fazer coisas: anotar o nome no caderno de visitantes, conferir se a pessoa pode entrar, ou barrar. Se está tudo certo, ele diz: pode passar. No Express, esse porteiro se chama middleware, que significa algo que fica no meio do caminho, entre o pedido chegar e a rota responder. Todo pedido passa pelos middlewares, na ordem, até chegar à rota.”

[1:45] Escrevendo um porteiro simples

Na tela: no server.js, logo depois de app.use(express.json());, escrever o código abaixo.

```
function registrarPedido(req, res, next) {
  console.log(`${req.method} ${req.originalUrl}`);
  next();
}

 app.use(registrarPedido);
```

Fala: “Vamos escrever o porteiro mais simples possível: o que anota cada visita. Um middleware é uma função com três nomes: req, o pedido; res, a resposta; e next, que quer dizer próximo. Dentro, escrevo no terminal o método e o endereço do pedido. Aquelas aspas inclinadas, com cifrão e chaves, deixam colocar valores dentro de um texto. E a linha mais importante: next, abre e fecha parênteses. É o porteiro dizendo: pode passar. Para colocar o porteiro na entrada, usamos app ponto use, que você já viu com o express ponto json: ele também é um middleware, que abre o envelope JSON. A ordem importa: o app ponto use vem antes das rotas.”

[3:30] Testando

Na tela: reiniciar o servidor, fazer alguns pedidos e mostrar o terminal.

GET /livros
POST /livros
GET /livros/1
Fala: “Reinicio o servidor e faço alguns pedidos. Olha o terminal: cada pedido foi anotado, com o método e o endereço. Isso é muito útil para descobrir o que está chegando na nossa API.”

[4:15] Sem o next, o pedido trava

Na tela: comentar a linha `next();`, reiniciar o servidor e abrir `/livros` no navegador, que fica carregando. Depois, restaurar a linha.

Fala: “Agora um erro que quase todo mundo comete. Vou apagar o next e reiniciar. Abro barra livros: a página fica carregando, carregando... O porteiro anotou o nome, mas não deixou passar nem barrou, e a pessoa ficou parada na porta para sempre. Regra: todo middleware ou chama next, ou responde ao cliente. Nunca fica sem fazer nenhum dos dois.”

[5:15] Barrar também é função do porteiro

Na tela: um esquema simples com duas setas saindo do porteiro: “next(): pode passar” e “res.status(400): barrado”.

Fala: “O porteiro também pode barrar. Quando ele barra, responde ao cliente e não chama next. A rota nem fica sabendo que o pedido existiu. É assim que vamos fazer a validação no próximo vídeo.”

[5:40] Resumo

Na tela: três frases curtas.

Fala: “Resumindo: middleware é uma função que fica entre o pedido e a rota. Ela pode olhar o pedido, deixar passar com next ou responder e barrar. E a ordem em que eles aparecem no arquivo importa. Até o próximo vídeo!”

Exercício sugerido: alterar o registrador para mostrar também a hora do pedido (com `new Date().toLocaleTimeString()`).